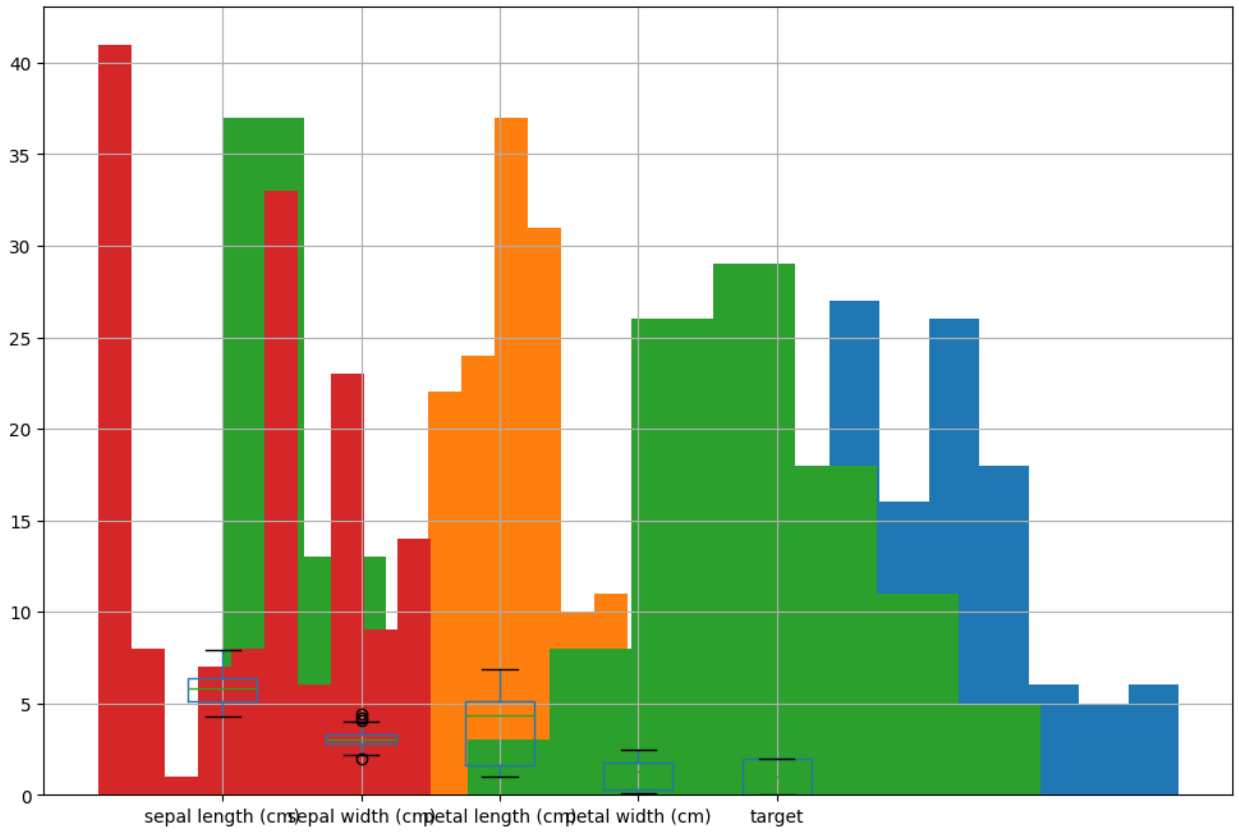
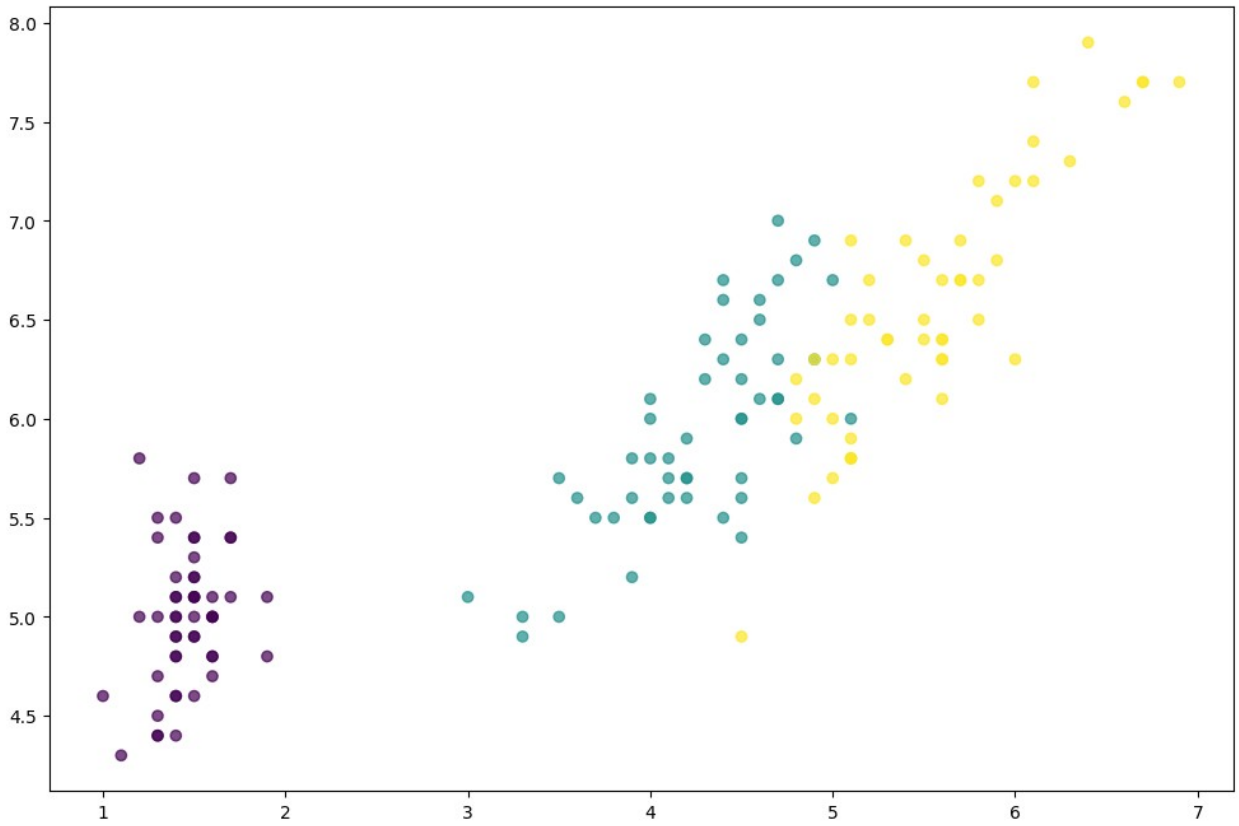
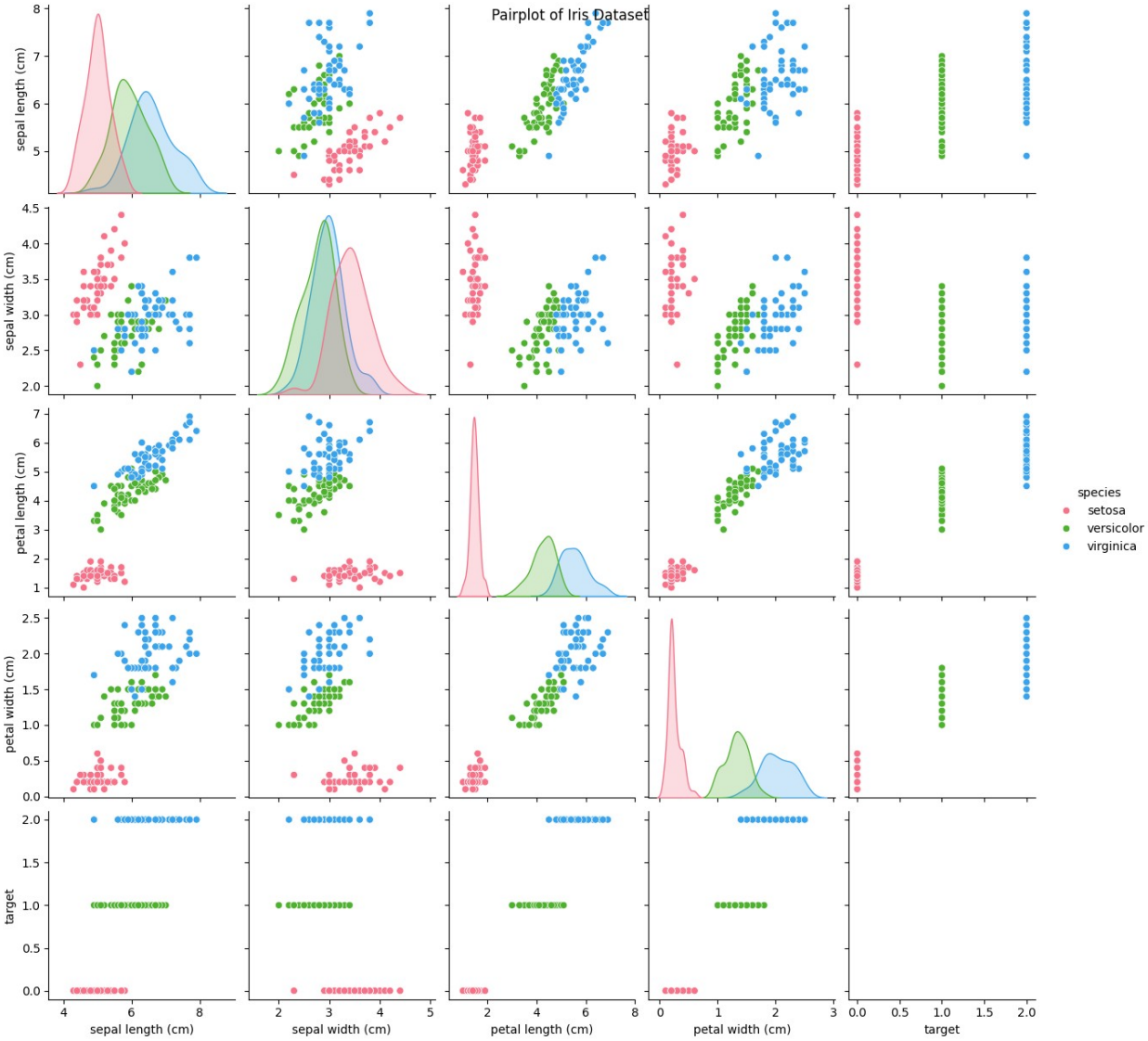
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
iris=load_iris()
iris_df=pd.DataFrame(iris.data,columns=iris.feature_names)
iris_df['target']=iris.target
iris_df['species']=iris_df['target'].map({0:'setosa',1:'versicolor',2:
'virginica'})
print("Iris Dataset Exploration")
print("Dataset Shape")
print(iris_df.shape)
print("Dataset 5 rows")
print(iris_df.head())
print("Dataset data types")
print(iris_df.dtypes)
print("Missing values")
print(iris_df.isnull().sum())
print("Class Distribution")
print(iris_df['species'].value_counts())

features=iris.feature_names
plt.figure(figsize=(12,8))
fig.suptitle("Histogram of Iris features")
for i,feature in enumerate(features):
    row,col=divmod(i,2)
    iris_df[feature].hist()

iris_df.boxplot()

plt.figure(figsize=(12,8))
scatter=plt.scatter(iris_df['petal length (cm)'],iris_df['sepal length
(cm)'],c=iris_df['target'],alpha=0.7,cmap='viridis')
plt.show()

plt.figure(figsize=(12,8))
sns.pairplot(iris_df,hue='species',palette='husl',height=2.5)
plt.suptitle("Pairplot of Iris Dataset")
plt.show()

plt.figure(figsize=(12,8))
corr=iris_df[features].corr()
sns.heatmap(corr,annot=True,cmap="coolwarm",center=0)
plt.suptitle("Correlation Matrix")
plt.show()

Iris Dataset Exploration
Dataset Shape
(150, 6)
```

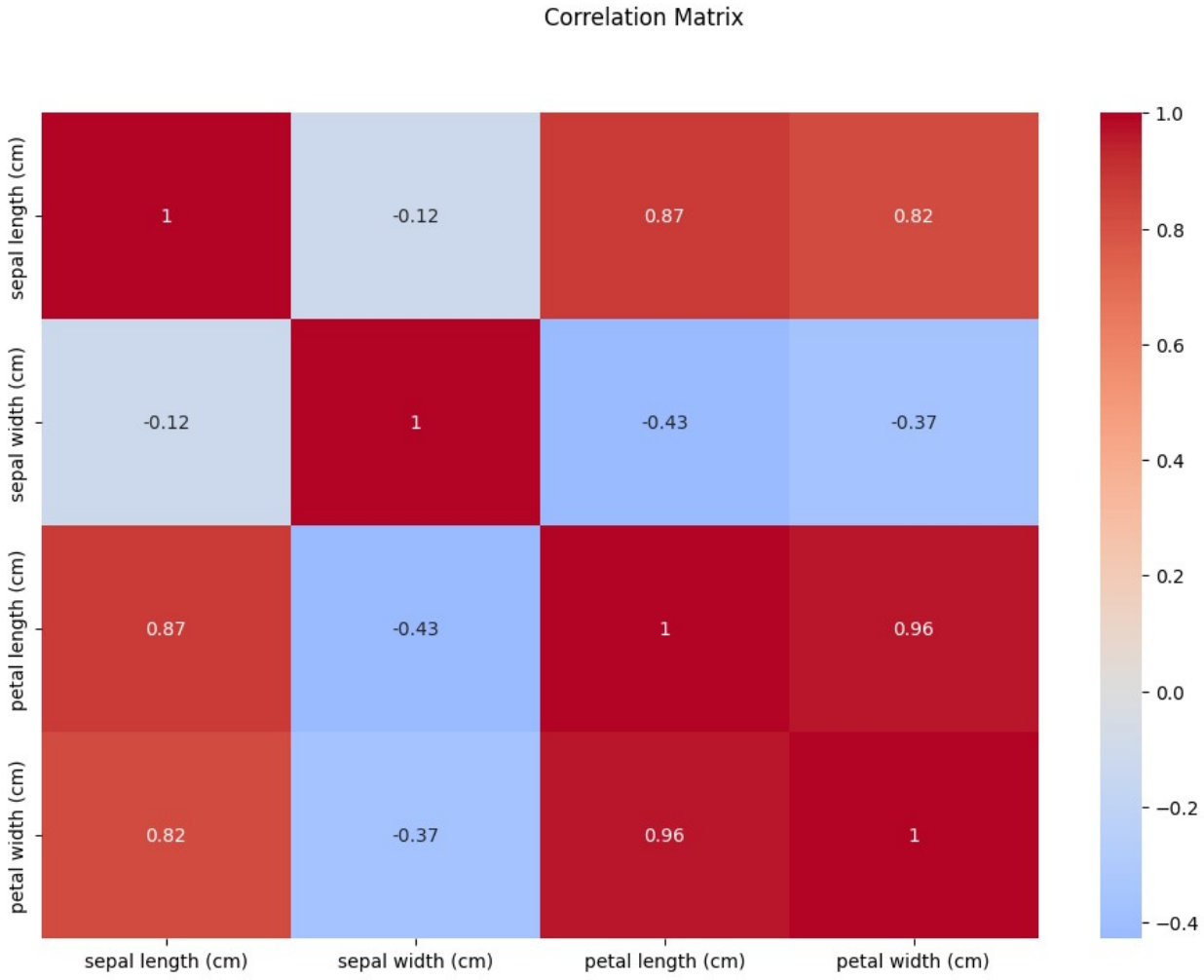
```
Dataset 5 rows
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width
(cm)  \
0                5.1               3.5                1.4
0.2
1                4.9               3.0                1.4
0.2
2                4.7               3.2                1.3
0.2
3                4.6               3.1                1.5
0.2
4                5.0               3.6                1.4
0.2

   target species
0       0  setosa
1       0  setosa
2       0  setosa
3       0  setosa
4       0  setosa
Dataset data types
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
target                 int64
species               object
dtype: object
Missing values
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
target               0
species              0
dtype: int64
Class Distribution
species
setosa        50
versicolor    50
virginica     50
Name: count, dtype: int64
```


40
35
30
25
20
15
10
5
0
sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)
target


8.0
7.5
7.0
6.5
6.0
5.5
5.0
4.5
1
2
3
4
5
6
7

```
<Figure size 1200x800 with 0 Axes>
```

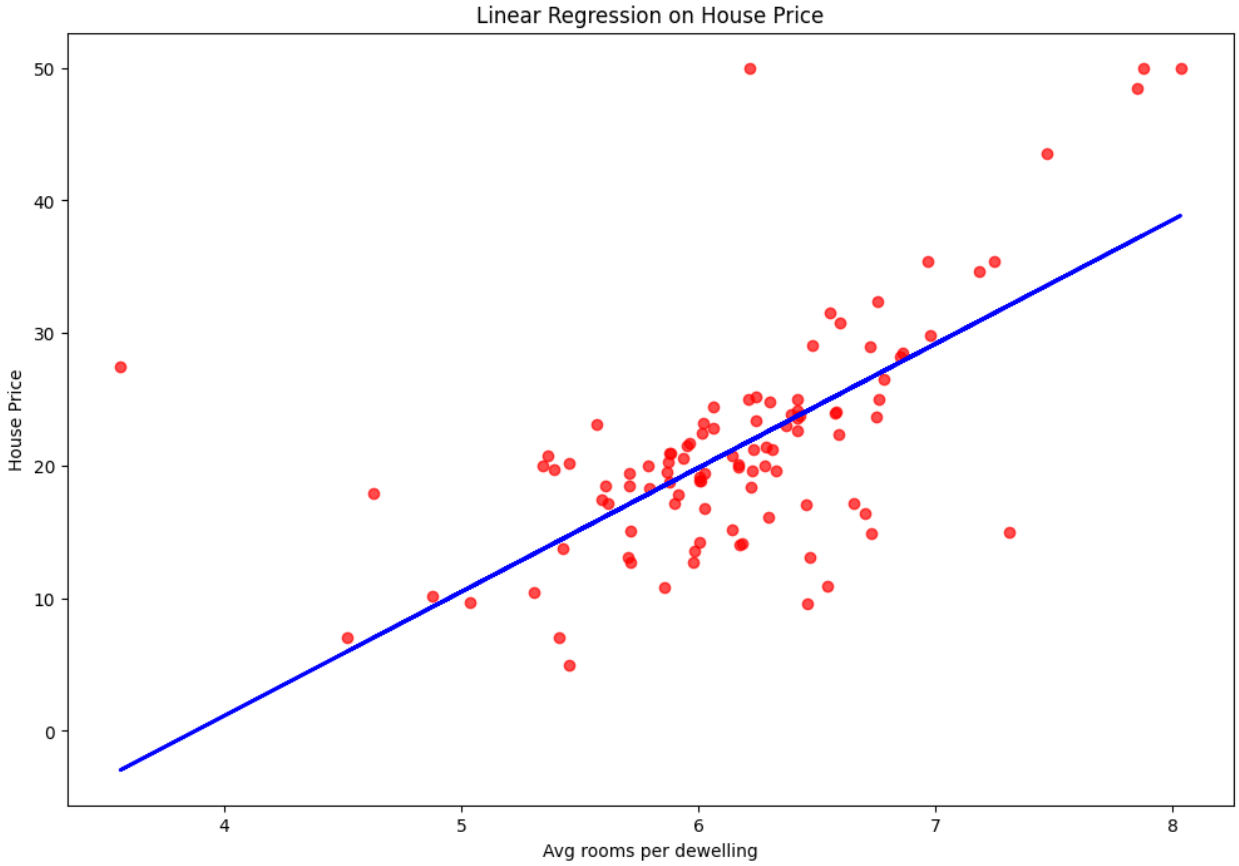


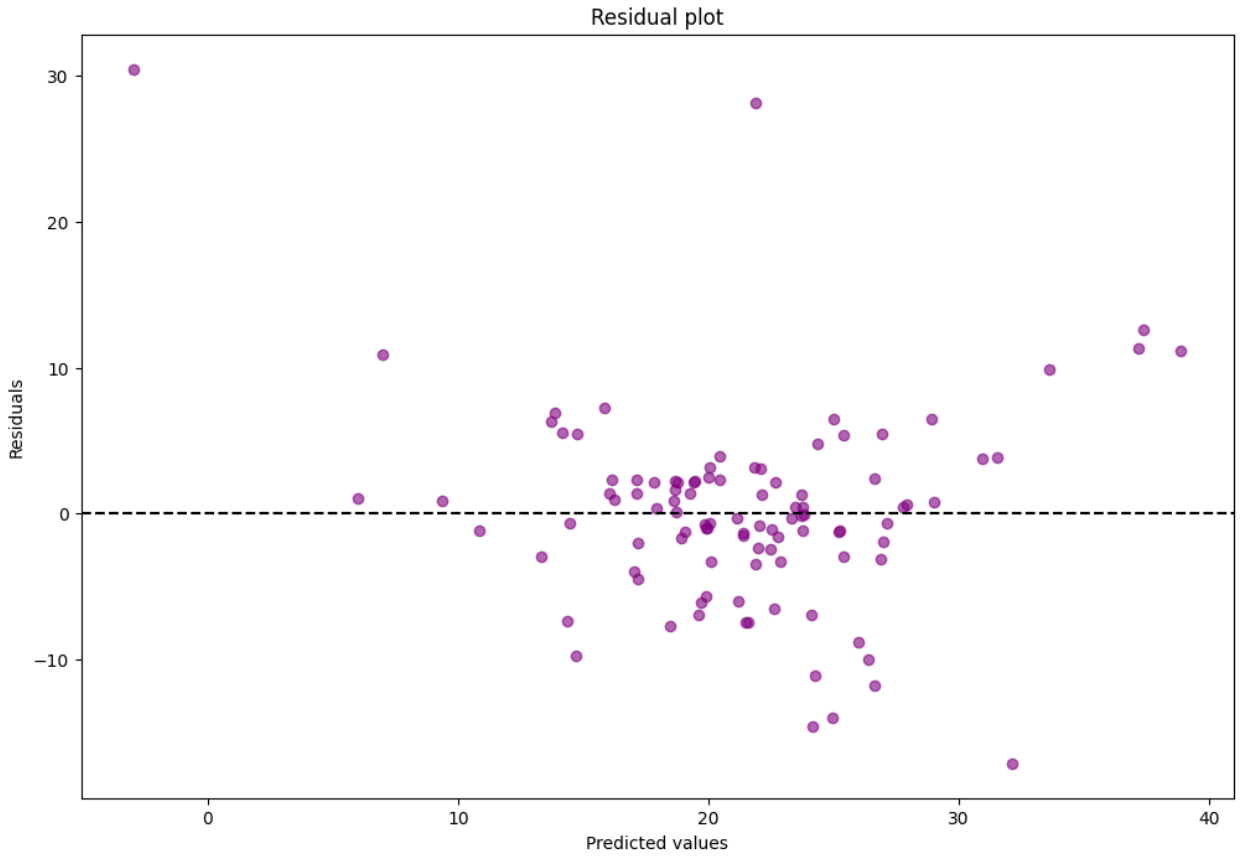



```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error,r2_score
boston=fetch_openml(name='boston',version=1,as_frame=True)
X=boston.data[["RM"]]
y=boston.target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,rando
m_state=42)
model=LinearRegression()
model.fit(X_train,y_train)
y_pred=model.predict(X_test)
print("MSE:",mean_squared_error(y_test,y_pred))
print("R^2 score:",r2_score(y_test,y_pred))
plt.figure(figsize=(12,8))
plt.scatter(X_test,y_test,color="red",alpha=0.7,label='actual')
```

```
plt.plot(X_test,y_pred,color="blue",linewidth=2,label="REgression
line")
plt.xlabel("Avg rooms per dewelling")
plt.ylabel("House Price")
plt.title("Linear Regression on House Price")
plt.show()

res=y_test-y_pred
plt.figure(figsize=(12,8))
plt.scatter(y_pred,res,color="purple",alpha=0.6)
plt.axhline(y=0,color="black",linestyle="--")
plt.xlabel("Predicted values")
plt.ylabel("Residuals")
plt.title("Residual plot")
plt.show()
```

```
MSE: 46.144775347317264
R^2 score: 0.3707569232254778
```

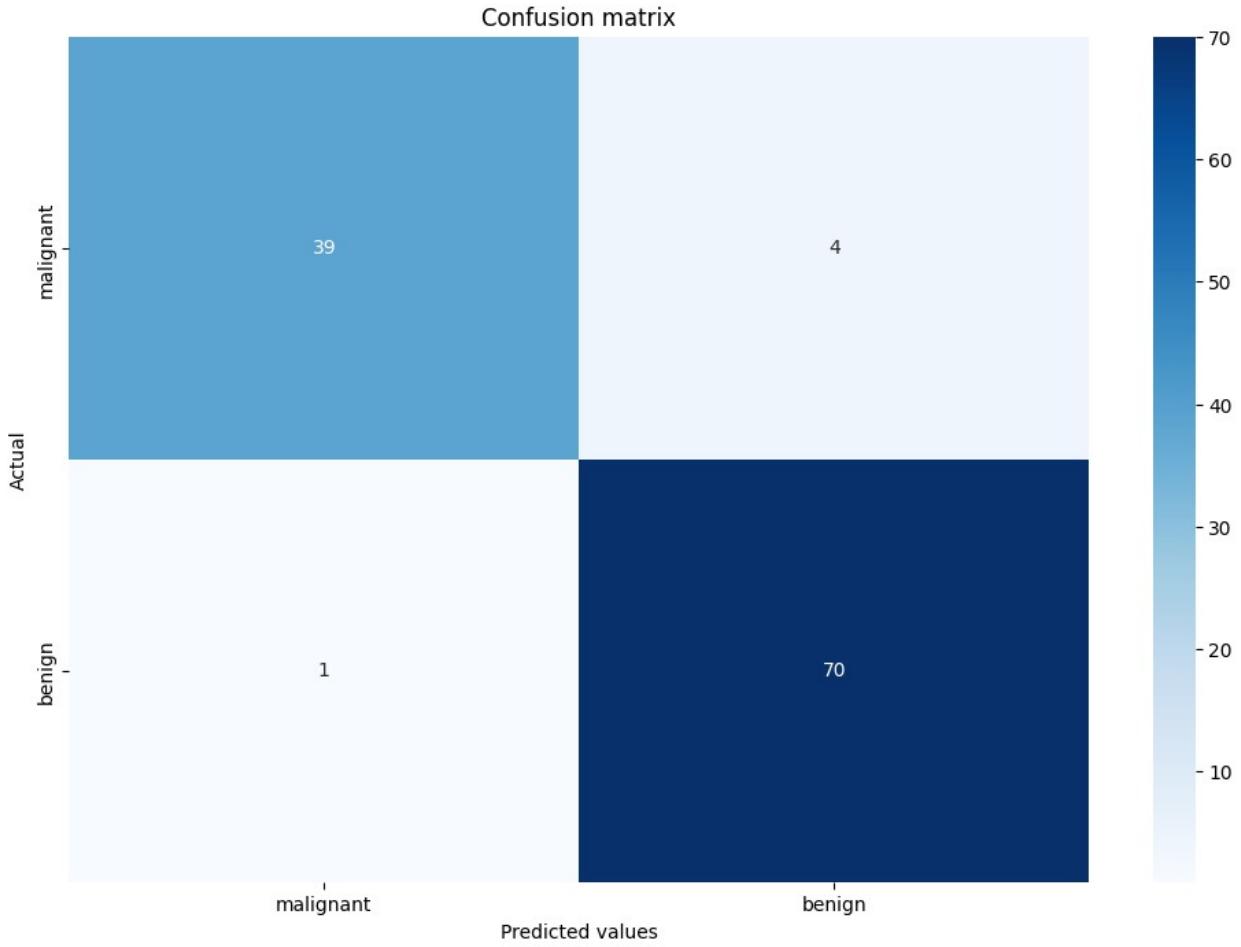





```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import
accuracy_score,precision_score,recall_score,confusion_matrix,classific
ation_report

data=load_breast_cancer()
X=data.data
y=data.target

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,rando
m_state=42)
model=LogisticRegression(max_iter=5000)
model.fit(X_train,y_train)
y_pred=model.predict(X_test)

print("Accuracy Score:",accuracy_score(y_test,y_pred))
print("Precision Score:",precision_score(y_test,y_pred))
print("Recall Score:",recall_score(y_test,y_pred))
```

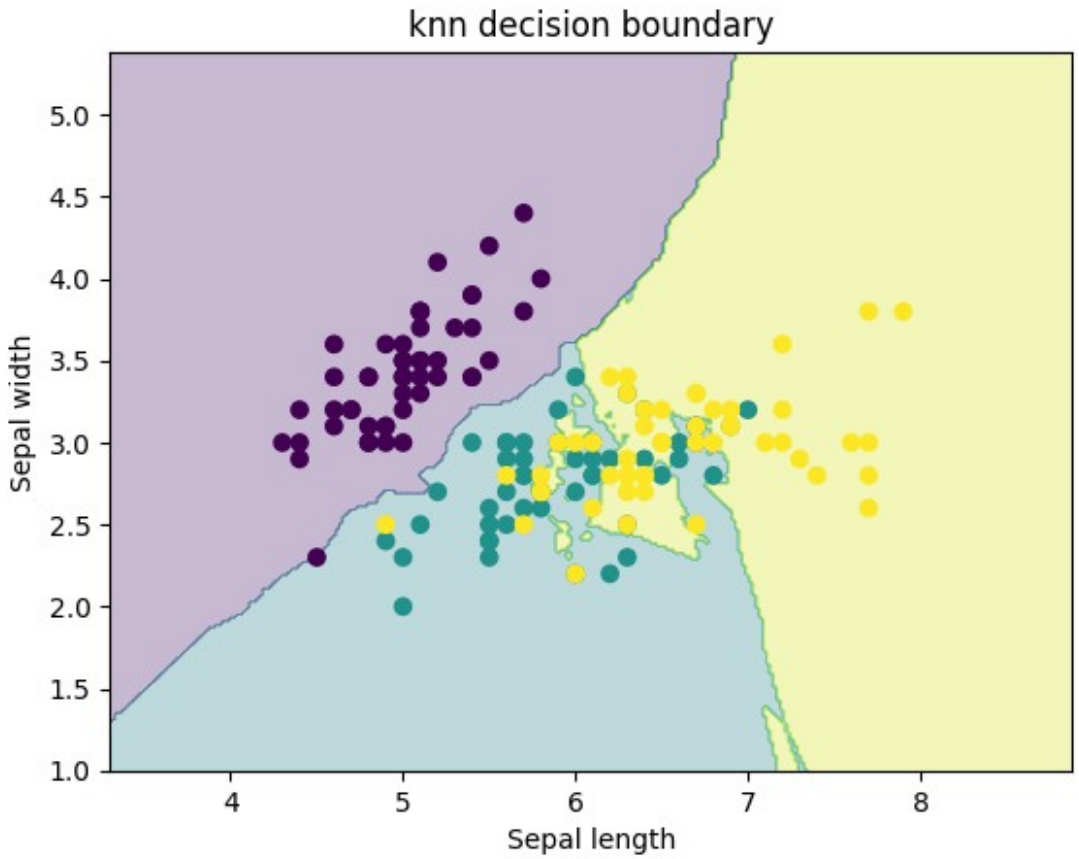
```
print("Classificationn Report:",classification_report(y_test,y_pred))

cm=confusion_matrix(y_test,y_pred)
plt.figure(figsize=(12,8))
sns.heatmap(cm,annot=True,cmap="Blues",fmt='d',xticklabels=data.target
_names,yticklabels=data.target_names)
plt.xlabel("Predicted values")
plt.ylabel("Actual")
plt.title("Confusion matrix")
plt.show()
```

```
Accuracy Score: 0.956140350877193
Precision Score: 0.9459459459459459
Recall Score: 0.9859154929577465
Classificationn Report:               precision    recall  f1-score
support

           0       0.97      0.91      0.94        43
           1       0.95      0.99      0.97        71

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114
```




```
from sklearn.neighbors import KNeighborsClassifier
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris=load_iris()
X = iris.data[:, :2]
y=iris.target

X_min,X_max=X[:,0].min()-1,X[:,0].max()+1
y_min,y_max=X[:,1].min()-1,X[:,1].max()+1

xx,yy=np.meshgrid(np.arange(X_min,X_max,0.02),np.arange(y_min,y_max,0.
02))
k=5
model=KNeighborsClassifier(n_neighbors=k).fit(X,y)
Z=model.predict(np.c_[xx.ravel(),yy.ravel()]).reshape(xx.shape)

plt.contourf(xx,yy,Z,alpha=0.3)
plt.scatter(X[:,0],X[:,1],c=y)
plt.xlabel("Sepal length")
```

```
plt.ylabel("Sepal width")
plt.title("knn decision boundary")
plt.show()
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,rando
m_state=42)
for k in [1,3,5,7,8]:
    m=KNeighborsClassifier(n_neighbors=k).fit(X_train,y_train)
    print(f"k={k} Accuracy:
{accuracy_score(y_test,m.predict(X_test)):.3f}")
```

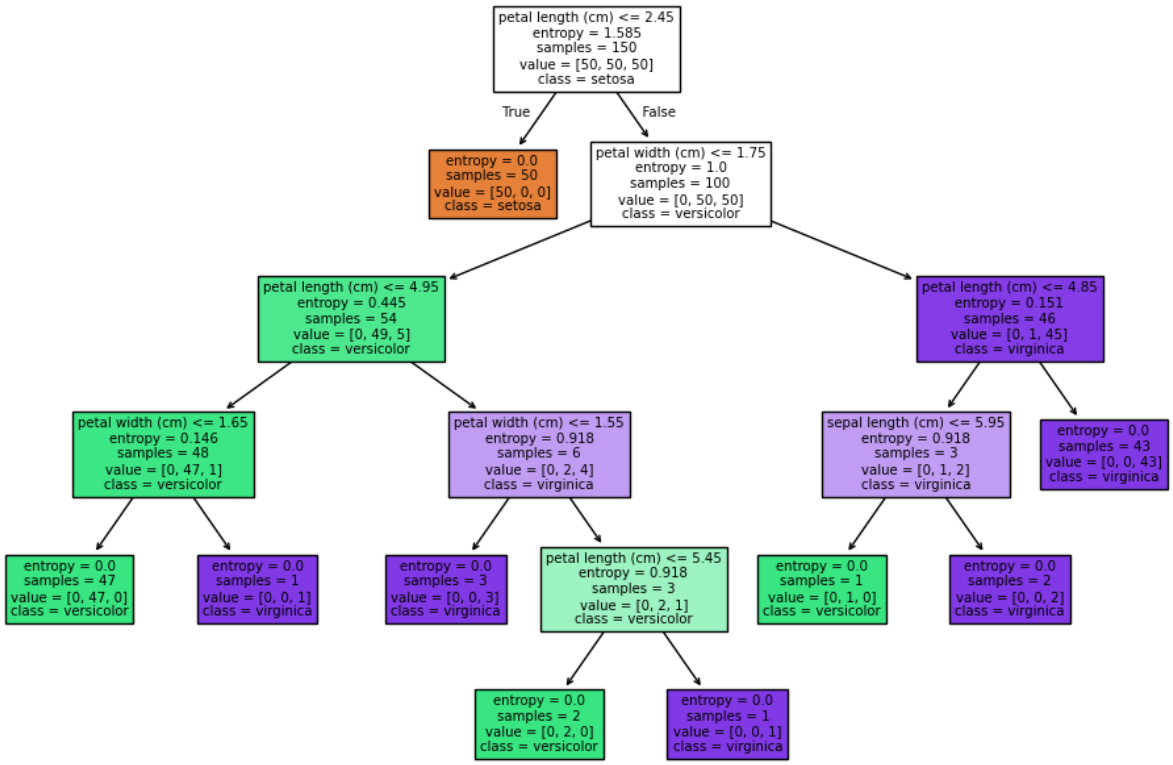



```
k=1 Accuracy:0.733
k=3 Accuracy:0.800
k=5 Accuracy:0.800
k=7 Accuracy:0.767
k=8 Accuracy:0.800
```

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier,plot_tree,export_text
import matplotlib.pyplot as plt

iris=load_iris()
X=iris.data
```

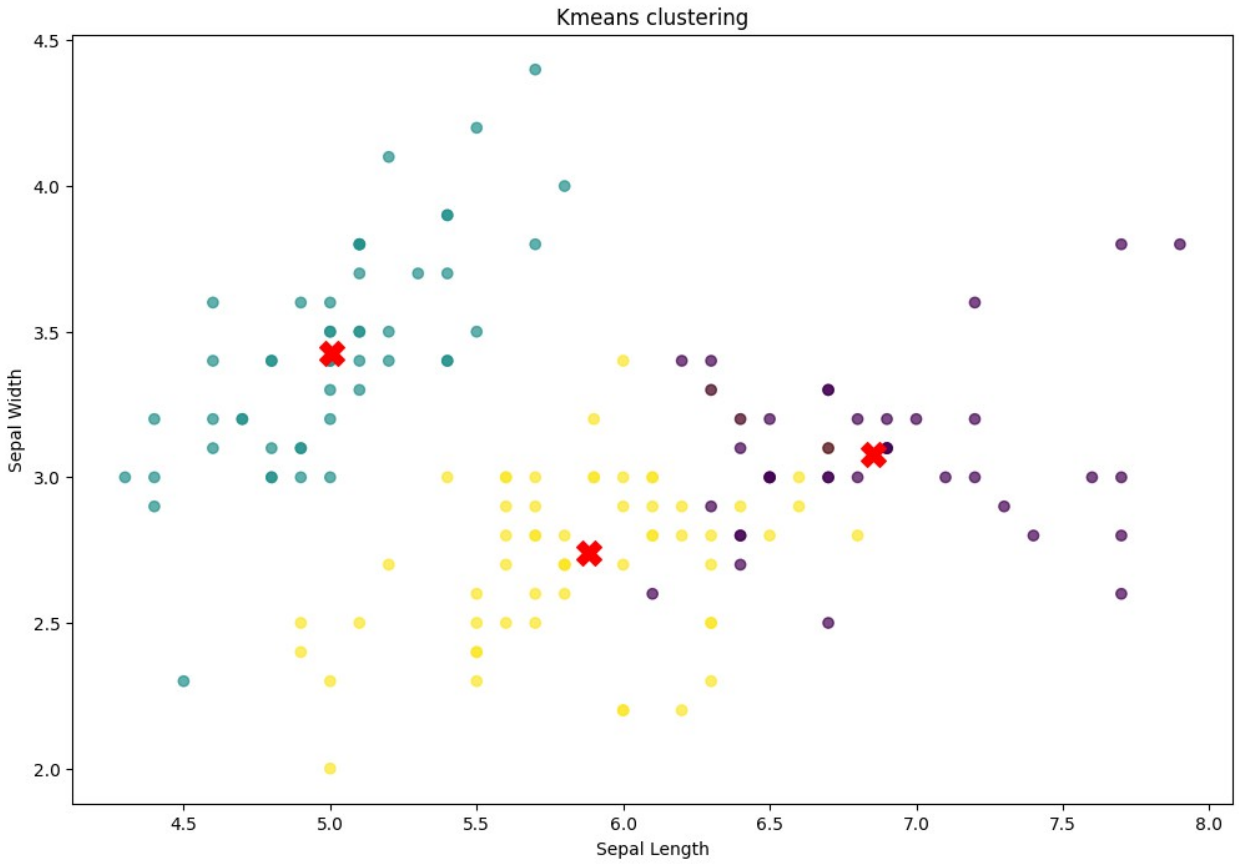
```
y=iris.target

print("First 5 rows")
for i in range(5):
    print(iris.feature_names,X[i])

model=DecisionTreeClassifier(criterion="entropy").fit(X,y)

plt.figure(figsize=(12,8))
plot_tree(model,feature_names=iris.feature_names,class_names=iris.targ
et_names,filled=True)
plt.show()

print("Accuracy:",model.score(X,y))
print("Decision Rules")
print(export_text(model,feature_names=iris.feature_names))
```

```
First 5 rows
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal
width (cm)'] [5.1 3.5 1.4 0.2]
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal
width (cm)'] [4.9 3.  1.4 0.2]
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal
width (cm)'] [4.7 3.2 1.3 0.2]
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal
width (cm)'] [4.6 3.1 1.5 0.2]
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal
width (cm)'] [5.  3.6 1.4 0.2]
```



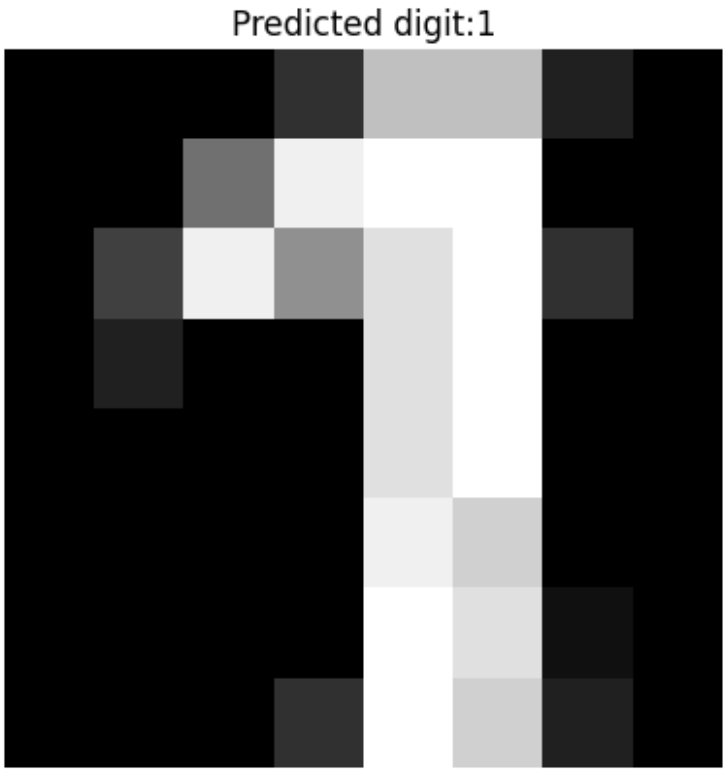

```
Accuracy: 1.0
Decision Rules
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.75
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- petal width (cm) <= 1.65
|   |   |   |   |--- class: 1
|   |   |   |--- petal width (cm) >  1.65
|   |   |   |   |--- class: 2
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- petal width (cm) <= 1.55
|   |   |   |   |--- class: 2
|   |   |   |--- petal width (cm) >  1.55
|   |   |   |   |--- petal length (cm) <= 5.45
|   |   |   |   |   |--- class: 1
|   |   |   |   |--- petal length (cm) >  5.45
|   |   |   |   |   |--- class: 2
|   |--- petal width (cm) >  1.75
|   |   |--- petal length (cm) <= 4.85
|   |   |   |--- sepal length (cm) <= 5.95
|   |   |   |   |--- class: 1
|   |   |   |--- sepal length (cm) >  5.95
```

```
|   |   |   |   |--- class: 2
|   |   |--- petal length (cm) >  4.85
|   |   |   |--- class: 2


from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

iris=load_iris()
X=iris.data
print("Data shape:",X.shape)

kmeans=KMeans(n_clusters=3,random_state=42)

kmeans.fit(X)

labels=kmeans.labels_
centers=kmeans.cluster_centers_
print("Centers:",centers)

plt.figure(figsize=(12,8))
plt.scatter(X[:,0],X[:,1],c=labels,cmap="viridis",alpha=0.7)
plt.scatter(centers[:,0],centers[:,1],s=200,marker='X',color='red')
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("Kmeans clustering")
plt.show()

Data shape: (150, 4)
Centers: [[6.85384615 3.07692308 5.71538462 2.05384615]
 [5.006      3.428      1.462      0.246     ]
 [5.88360656 2.74098361 4.38852459 1.43442623]]
```




```
from sklearn.datasets import load_digits
from sklearn.svm import SVC
import matplotlib.pyplot as plt

digits=load_digits()
X=digits.data
y=digits.target

X_train=X[:1500]
y_train=y[:1500]
X_test=X[1500:]
y_test=y[1500:]

model=SVC(gamma=0.01)
model.fit(X_train,y_train)

pred=model.predict([X_test[0]])

plt.imshow(digits.images[1500],cmap='grey')
plt.title(f"Predicted digit:{pred[0]}")
plt.axis("off")
plt.show()
```

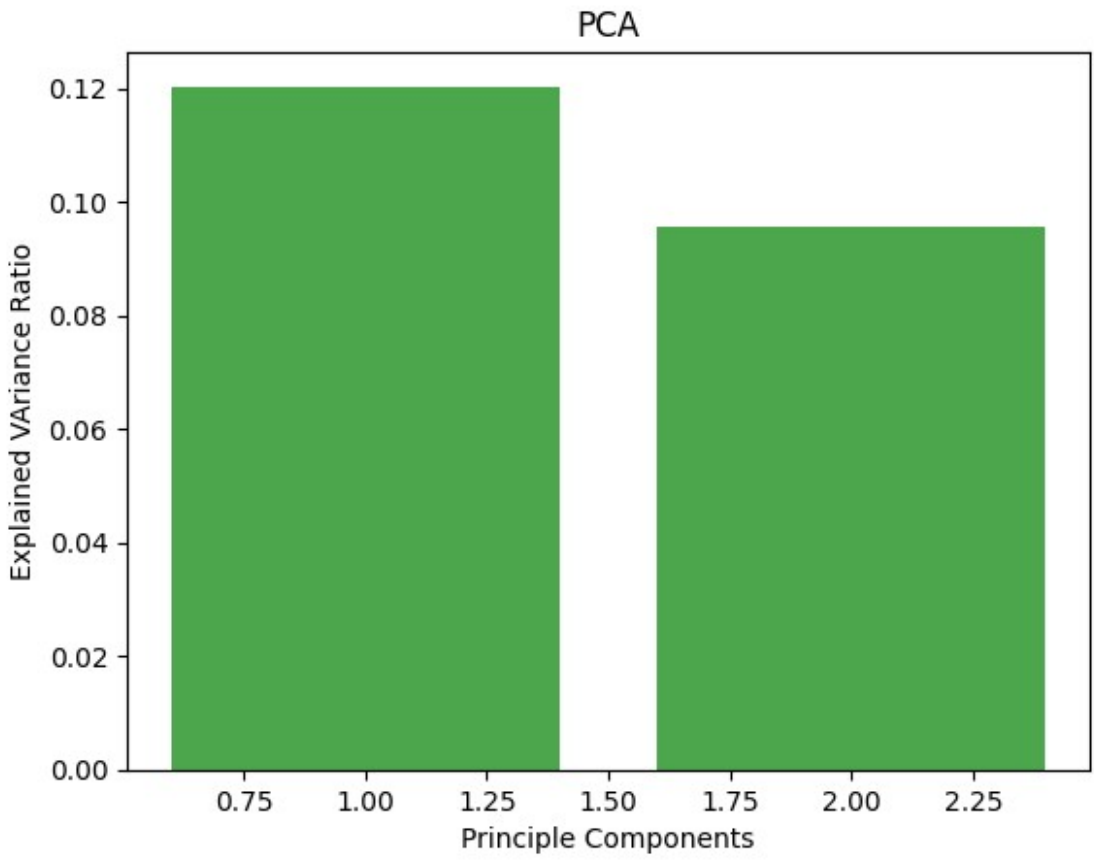
```
acc=model.score(X_test,y_test)
print("Accuracy of the model:",acc)
```

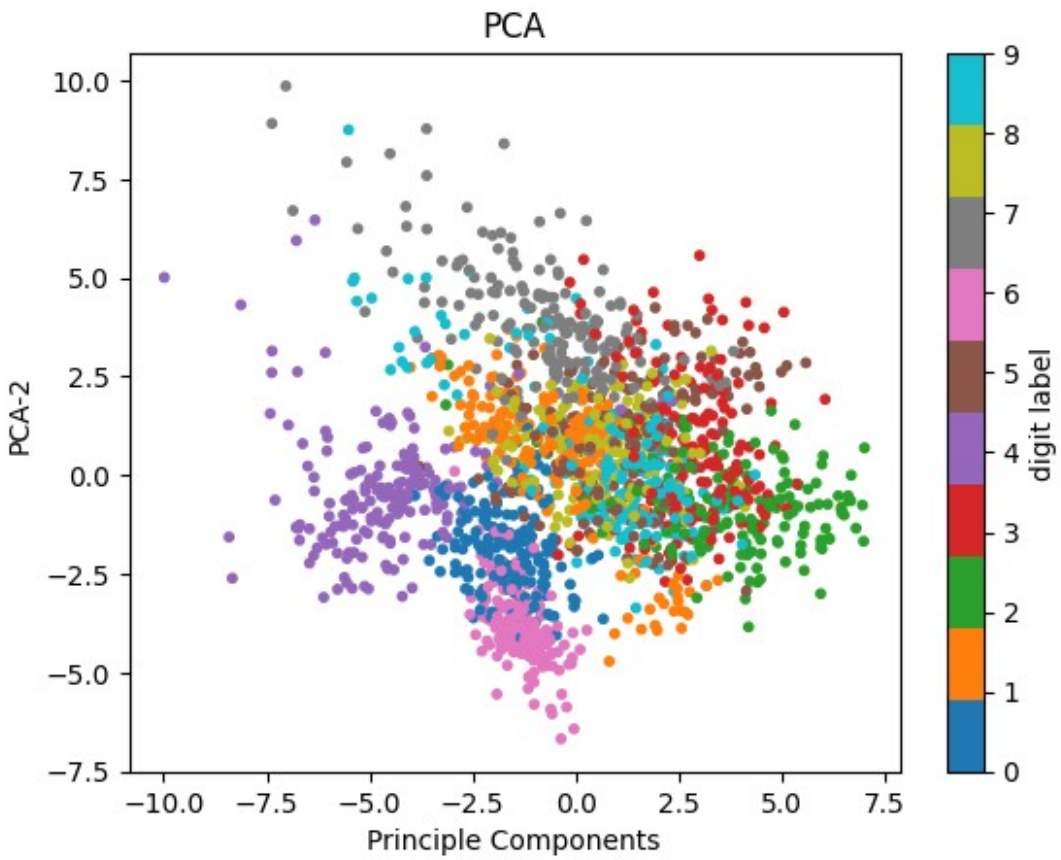



```
Accuracy of the model: 0.6902356902356902
```

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

digits=load_digits()
X=digits.data
y=digits.target

scaler=StandardScaler()
X_scaled=scaler.fit_transform(X)

pca=PCA(n_components=2)
X_pca=pca.fit_transform(X_scaled)

plt.bar([1,2],pca.explained_variance_ratio_,color="green",alpha=0.7)
plt.xlabel("Principle Components")
plt.ylabel("Explained VAriance Ratio")
plt.title("PCA")
plt.show()
```

```
plt.scatter(X_pca[:,0],X_pca[:,1],c=y,cmap="tab10",s=10)
plt.xlabel("Principle Components")
plt.ylabel("PCA-2")
plt.title("PCA")
plt.colorbar(label="digit label")
plt.show()
```




PCA
PCA-2
Principle Components
digit label
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
9
8
7
6
5
4
3
2
1
0